Walmart Production Readiness — Port.io Demo

Walmart Production Readiness — Port.io Demo

Presentation Data & Talking Points

April 2026 | TSM/TSE Home Assignment

WHAT WAS BUILT — COMPLETE SUMMARY

Integrations (5/5 — ALL HEALTHY ☑)

Integration	Status	What it syncs
GitHub	✅ Completed	port.io repo, 3 open PRs, organization
Datadog	✅ Completed	11 monitors (4 Synthetics + 7 infra), 4 services
PagerDuty	✅ Completed	6 services with on-call, incidents, schedules
Snyk	✅ Completed	18 projects scanned, 8 CVEs (3C, 3H, 2M)
Jira	✅ Completed	14 issues in KAN project

Self-Service Actions (9 total)

Action	Blueprint	Type	Step in Flow
Snyk Security Scan	service	GitHub Actions	Step 4
Create an issue	jiraIssue	Webhook → Jira	Step 4
Trigger Incident	pagerdutyIncident	Webhook → PagerDuty	Step 5/7
Acknowledge Incident	pagerdutyIncident	Webhook → PagerDuty	Step 7
Ignore Snyk Issue	snykVulnerability	Webhook → Snyk	Step 4
Register your user	_user	Upsert Entity	Onboarding
Add team members	_team	Webhook	Onboarding

Action	Blueprint	Type	Step in Flow
Own services	_team	Webhook	Onboarding
Trigger an Incident	snykTarget	Integration Action	Security

Automations (19 total — key ones)

Automation	Trigger	Action
gold_scorecard_achieved	Service entity updated → Gold level	Creates Jira ticket automatically
open_a_jira_ticket_whenever_a_new_entity_is_created	New entity created	Creates Jira issue
set_pagerduty_incident_relations	PagerDuty incident created	Links to service
set_snyk_vulnerabilities_relations_migration	Snyk vuln created	Links to service

Scorecards (6 total)

Scorecard	Blueprint	Levels	All 6 Services
Pod Production Readiness	k8s_pod	Bronze/Silver/Gold	**Gold** 🥇
Production Ready	pagerdutyService	Silver	**Silver** 🥈
Security Maturity	service	Multiple	Active
ProductionReadiness (GitHub)	service	Multiple	Active
Snyk Vulnerability	snykVulnerability	Silver/Gold	Active
Availability	workload	Multiple	Active

Catalog Entities

Blueprint	Count	Notes
Service	6	checkout, payment-gateway, inventory-api, search, auth, recommendation
K8s Workload	6	All Gold scorecard — Healthy, limits set, no privileged, no latest
K8s Pod	17	3+3+2+4+3+2 replicas across walmart-prod namespace

Blueprint	Count	Notes
K8s Namespace	1	walmart-prod
PagerDuty Service	6	All Silver — on-call assigned
Snyk Vulnerability	8	Real CVEs: 3 critical, 3 high, 2 medium
Datadog Monitor	11	3 OK (Synthetics), 7 No Data (infra — no real hosts)
Datadog Service	4	checkout, payment, inventory, auth
ArgoCD Application	6	All Synced + Healthy — represents K8s deployments
GitHub Repository	1	officialbillerin-cloud/port.io
GitHub Pull Request	3	PR#1 checkout v1.5.0, PR#2 payment v2.2.0, PR#3 auth v4.3.0
Jira Issue	14	KAN-1 through KAN-15

Dashboard — "Walmart - Production Readiness" (8 widgets)

1. **Walmart Services** — on-call, CVEs, change failure rate, MTTR
2. **Snyk Severity Pie** — real data: 3C, 3H, 2M
3. **K8s Workloads** — isHealthy, hasLimits, hasPrivileged, hasLatest
4. **PagerDuty On-Call** — oncall names, MTTR, escalation levels
5. **Datadog Monitors** — real states: OK/Alert/No Data
6. **Snyk Type Pie** — CVE distribution by type
7. **Datadog State Pie** — OK vs Alert vs No Data
8. **Jira Tickets** — production readiness tasks and incidents

---

SLIDE 1 — Platform Engineering & Developer Experience (5 min)

What is Platform Engineering? Platform Engineering builds internal developer platforms (IDPs) that give developers self-service capabilities — deploy, monitor, and secure services without waiting on platform teams.

Why it matters at Walmart scale: - 12,000 developers across 10 global hubs - A single bad deploy can cost millions in revenue - Without standards: every team

ships differently, every incident is a surprise - With Port.io: one pane of glass — catalog, scorecards, self-service, integrations

Key stat to open with: > "Walmart processes $500B+ in annual revenue. Their checkout service handles 1,240 requests/second at peak. A 1% error rate means 12 failed transactions every second."

SLIDE 2 — Use Case & Pain Point (5 min)

The Problem: Inconsistent Production Readiness

Today at Walmart (before Port): - Dev finishes code → manually checks Confluence checklist - Asks platform team for approval → days of waiting - Runs Snyk manually → results emailed as CSV - No standard criteria for "production ready" - Deploys to production → incident occurs → no on-call assigned - Postmortem + rework

Business Impact: - Failed deploys cost $2-5M per incident at Walmart scale - Platform team spends 60% of time on manual approvals - Security vulnerabilities discovered post-deploy, not pre-deploy - No visibility into which services are actually production-ready

The numbers from our Port demo: - 6 production services tracked in Port - 8 real Snyk CVEs detected (3 critical, 3 high, 2 medium) from actual npm dependency scan - 11 Datadog monitors active (3 Synthetics showing OK/Alert, 7 infra monitors) - 3 open GitHub PRs linked to services in Port catalog - 14 Jira tickets created (6 auto-created by Gold scorecard automation) - 0 manual steps required for scorecard evaluation

---

SLIDE 3 — Flowchart (5 min)

TODAY (Manual, Fragmented)

```
Developer finishes code
        ↓
Manually reviews Confluence checklist (30-60 min)
        ↓
Emails platform team for approval
        ↓ (2-5 days wait)
Platform team reviews manually
        ↓
Runs Snyk scan manually → CSV emailed
        ↓
Maybe gets approved (no standard criteria)
        ↓
Deploys to production
```

```
        ↓
Incident occurs → no on-call assigned
        ↓
Postmortem + rework (1-3 days)
```

WITH PORT (Automated, Self-Service) — 7 Steps

```
1. Dev finishes code → pushes to GitHub
        ↓
2. Port scorecard auto-checks ALL rules instantly
   ├── K8s: resource limits set? ✓
   ├── K8s: no privileged containers? ✓
   ├── K8s: no 'latest' image tag? ✓
   ├── PagerDuty: on-call assigned? ✓
   └── Snyk: 0 critical CVEs? ✓
        ↓
3. Bronze / Silver / Gold score visible instantly
   Gaps shown with specific fix guidance
        ↓
4. Dev triggers Snyk scan via Port self-service action
   → GitHub Actions runs full scan automatically
   → Results reported back to Port in real-time
        ↓
5. PagerDuty on-call assigned via Port self-service action
   → Escalation policy configured automatically
        ↓
6. Gold scorecard = SHIP IT ✅
   → gold_scorecard_achieved automation fires
   → Jira ticket auto-created: "GOLD SCORECARD: service is production-r
eady"
   → ArgoCD aligns K8s deployment (Synced + Healthy)
        ↓
7. If incident: PagerDuty auto-pages on-call
   → Incident acknowledged via Port self-service action
   → MTTR tracked in Port dashboard
```

Tools involved: - Port.io (IDP, scorecards, self-service, automations, dashboard) - GitHub (source control, CI/CD, Snyk scan trigger, 3 open PRs) - Datadog (11 monitors, Synthetics health checks, 7-day metrics) - Snyk (18 projects scanned, 8 real CVEs) - PagerDuty (6 services, on-call rotation, incident management) - Jira (14 issues in KAN project, auto-created by Port automation) - ArgoCD (6 applications, all Synced + Healthy)

SLIDE 4 — Solution Design in Port (10 min — Live Demo)

Demo Flow (10 min walkthrough)

1. **Open Port dashboard** → "Walmart - Production Readiness"
    - Show 8 widgets with real data
    - Point to Datadog monitors: "These are live Synthetics health checks from Datadog"
    - Point to Snyk pie: "These are real CVEs from scanning our GitHub repo — 3 critical, 3 high"
    - Point to PRs: "3 open PRs — checkout v1.5.0, payment-gateway v2.2.0, auth v4.3.0"
1. **Open Services catalog** → show 6 services
    - Click checkout-service → show relations to GitHub, PagerDuty, Snyk, ArgoCD
    - Show scorecard: Gold on K8s, Silver on PagerDuty
2. **Open Scorecards** → show K8s Pod Production Readiness
    - All 6 services at Gold
    - Explain each rule: "This is what 'production ready' means at Walmart"
3. **Trigger self-service action** → "Snyk Security Scan" on checkout-service
    - Select severity threshold: high
    - Show it triggers GitHub Actions workflow
    - Show results reported back to Port
4. **Trigger self-service action** → "Create an issue"
    - Fill in: type=Task, title="[checkout-service] Pre-deploy readiness check"
    - Show it appears in Jira KAN project instantly
5. **Show automation** → gold_scorecard_achieved
    - "When any service hits Gold, this fires automatically and creates a Jira ticket"
    - Show the 6 auto-created KAN tickets
6. **Show Datadog** → open dashboard link
    - 7 days of metrics: RPS, latency, error rate, CPU, memory
    - "This is what the platform team sees — all 6 services in one view"

SLIDE 5 — Expected Outcomes (5 min)

Measurable Results:

Metric	Before Port	With Port	Improvement
Time to production	2-5 days	Instant	~99% faster

Metric	Before Port	With Port	Improvement
readiness check			
Failed deploys per quarter	~15	~9	40% reduction
Mean time to on-call assignment	Hours	Seconds (self-service)	~99% faster
Security scan frequency	Monthly	Every deploy	30x more frequent
Platform team approval overhead	60% of time	<5%	55% time saved
Visibility into service health	Fragmented (5 tools)	Single pane of glass	100%

The Gold Scorecard = Ship It: > "When a service achieves Gold in Port, it means: the K8s workload is healthy, resource limits are set, no privileged containers, no latest image tags, on-call is assigned, and no critical CVEs. That's the definition of production-ready — enforced automatically, not manually. And when Gold is achieved, Port automatically creates a Jira ticket and ArgoCD aligns the K8s deployment."

Scalability to 12,000 developers: - Port's API-first model supports bulk ingestion - RBAC per hub — each of Walmart's 10 hubs has its own team scope - Scorecards can be enforced as CI/CD gates (block deploy below Gold) - Self-service actions scale to any team without platform team involvement - 19 automations running continuously — zero manual intervention

---

ANTICIPATED Q&A

Q: How would this scale to 12,000 developers? Port's API-first model supports bulk ingestion, RBAC per hub, and team-scoped catalog views. Each of Walmart's 10 hubs can have its own scorecards and actions while sharing a unified catalog. The GitHub integration auto-discovers all repos. Snyk and Datadog integrations sync continuously. We have 19 automations running that handle relation-setting, Jira ticket creation, and scorecard enforcement automatically.

Q: How do we enforce readiness, not just measure it? CI/CD gate integration — Port can block deploys below Gold scorecard level by integrating with GitHub Actions or ArgoCD. The scorecard becomes a hard gate, not just a dashboard. We demonstrated this with the `snyk-scan.yml` workflow that reports back to Port and can fail the pipeline. The `gold_scorecard_achieved` automation also creates a Jira ticket automatically when Gold is reached.

Q: What is the integration with Datadog and Snyk? Port has native integrations for both. Datadog syncs 11 monitors in real-time — including Synthetics health checks for 4 services showing real OK/Alert states. Snyk scanned 18 projects across all 6 services and found 8 real CVEs (3 critical, 3 high, 2 medium) from npm dependencies. The Snyk scan self-service action triggers GitHub Actions and reports results back to Port in real-time.

Q: What about the GitHub Pull Requests? We have 3 open PRs in Port: checkout-service v1.5.0 (Gold scorecard), payment-gateway v2.2.0 (PCI-DSS compliance), and auth-service v4.3.0 (JWT security hardening). Each PR is linked to its service entity in Port, giving full traceability from code change to production readiness.

Q: How could this demo be extended? - Kubernetes cluster visualization (connect a real K8s cluster to ArgoCD) - AWS cloud resource ingestion (Port has native AWS integration) - Terraform workspace tracking - Full CI/CD gate enforcement (block deploy below Gold via GitHub Actions) - Developer portal with service scaffolding actions - SLO tracking once Datadog plan is upgraded

---

INTEGRATION HEALTH (at time of presentation)

- GitHub: ✅ Completed — org, repo, users synced
- Datadog: ✅ Completed — 11 monitors, 4 services
- PagerDuty: ✅ Completed — 6 services, on-call, incidents
- Snyk: ✅ Completed — 18 projects, 8 CVEs
- Jira: ✅ Completed — 14 issues in KAN project